

420-301-VA: Web Services

Lab 1: Socket Programming in Java

Computer Science & Technology
Vanier College

Introduction

In our tutorial, we explored the fundamentals of network communication using the Transmission Control Protocol (TCP), where we saw how applications rely on IP addresses to find machines and Port numbers to locate specific applications.

This lab will guide you through implementing the core architectures of network programming:

- **One-Way and Two-Way Communication:** Using the `Socket` and `ServerSocket` classes to establish a reliable TCP connection, and wrapping byte streams into character streams (`BufferedReader` and `PrintWriter`) to exchange text.
- **Multi-threaded Servers:** Moving beyond a simple blocking server to a concurrent architecture. You will use threads to allow a single server to handle multiple clients simultaneously without freezing, which is the foundational pattern for modern web servers.

Objectives

This lab provides a hands-on introduction to building networked applications in Java. By the end of this lab, you should be comfortable with:

- Binding a `ServerSocket` to a specific port to listen for incoming traffic.
- Connecting a `Socket` client to a remote server's IP and port.
- Extracting and using `InputStream` and `OutputStream` for data transfer.
- Understanding the “blocking” nature of the `accept()` method.
- Implementing the `Runnable` interface to delegate client handling to separate threads.

Instructions

- Please read the companion tutorial and try out the examples first to deepen your understanding.
- For each exercise, create the required classes. You will need separate terminals (or split terminals in VS Code) to run the Server and Client independently.
- Copy-paste your formatted code for each exercise into a separate document, followed by screenshots of your terminal outputs to validate that your code worked as expected.
- You can work with your peers, but each person must write, execute, and document their own work.

1 Part 1: Basic Socket Communication

1.1 Concept Review: The Client-Server Handshake

To establish a TCP connection, the Server must act first by opening a port and waiting. We use `ServerSocket.accept()` for this, which pauses the server's execution until a client connects. The Client then uses `new Socket(ip, port)` to initiate the connection.

Once connected, both sides look at the connection as a pipe. Data written to the `OutputStream` on one side is read from the `InputStream` on the other side.

```
// To READ text easily (line by line) from the socket:
BufferedReader in =
new BufferedReader(new InputStreamReader(socket.getInputStream()));
String message = in.readLine(); // Blocks until a line of text is received

// To WRITE text easily (and auto-flush) to the socket:
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
out.println("Hello from the other side!"); // Sends data over the network
```

Listing 1: Example: Wrapping Streams for Text Communication

Exercise 1: IoT Temperature Logging System

Imagine you are building a backend for Internet of Things (IoT) devices. You have a remote temperature sensor (the Client) that wakes up, connects to your main server, sends a single temperature reading, and then disconnects to save battery. The Server just listens, prints the reading, and waits for the next connection.

1. The Server: Create a class named `TemperatureServer`.

- In the `main` method, create a `ServerSocket` listening on port 8000. Use a `try-with-resources` block.
- Create an infinite `while (true)` loop. Inside the loop:
 - (a) Call `accept()` to wait for a client.
 - (b) Print “[SERVER] Sensor connected!”
 - (c) Wrap the client’s `InputStream` in a `BufferedReader`.
 - (d) Read one line of text using `readLine()` and print it to the console (e.g., “[SERVER] Received data: [data]”).
 - (e) Close the `clientSocket` (do not close the `serverSocket`).

2. The Client: Create a class named `TemperatureSensor`.

- In the `main` method, create a `Socket` connecting to “localhost” on port 8000. Use a `try-with-resources` block.
- Wrap the socket’s `OutputStream` in a `PrintWriter` (remember to pass `true` for auto-flushing).
- Generate a random temperature between 10.0 and 30.0.
- Send a string formatted like: “Sensor-A1: 22.5C” using `println()`.
- Print “[SENSOR] Data transmitted successfully.” to the client’s local console.

3. Test your System:

- Open a terminal and run your server: `java TemperatureServer`
- Open a second terminal and run your client: `java TemperatureSensor`
- Run the client a few more times. You should see the server logging multiple different temperature readings sequentially.

4. Observe the Results:

- The Server runs indefinitely, processing one sensor connection at a time. The Client terminates immediately after sending its data.

5. Thinking Question:

- Kill the server process using `Ctrl+C` in its terminal. Now, try to run the `TemperatureSensor` client again. What exact `Exception` is thrown? Why does this happen, and how does it relate to the “postal service” analogy from the tutorial?

2 Part 2: Multi-threading for Concurrency (Behavioral)

2.1 Concept Review: Breaking the Bottleneck

The server you built in Exercise 1 is a **blocking** server. If a client connects but sends data very slowly, the server gets “stuck” waiting for that data. If a second client tries to connect during this time, they must wait.

To solve this, we use **Threads**. The main server thread’s *only* job is to sit at the door (`accept()`). When a client arrives, it hands the client’s `Socket` to a new worker thread (a `ClientHandler`), and immediately goes back to the door to wait for the next client.

```
while (true) {  
    Socket clientSocket = serverSocket.accept();  
    System.out.println("New client connected!");  
  
    // Hand the socket to a Runnable handler and start a new Thread  
    ClientHandler handler = new ClientHandler(clientSocket);  
    new Thread(handler).start();  
}
```

Listing 2: Example: Spawning Threads for Clients

Exercise 2: Concurrent Calculator Service

In this exercise, you will build a two-way `CalculatorServer` that can handle math requests from multiple users at the exact same time. The client will send a math problem (like “5 + 10”), and the server will compute the answer and send it back.

1. **The ClientHandler (Runnable):** Create a class named `CalcClientHandler` that implements `Runnable`.
 - It needs a `private Socket socket` field, initialized via a constructor.
 - In the `run()` method, wrap both the `InputStream` and `OutputStream`.
 - Send a welcome message to the client: “Welcome to the Math Server! Type ‘quit’ to exit.”
 - Create a `while` loop to read lines from the client continuously.
 - If the text is “quit”, break the loop.
 - Otherwise, assume the text is formatted with spaces (e.g., “10 + 5”). Use `String.split(" ")` to break it into three parts.
 - Parse the numbers, check the operator (+, -, *, /), perform the calculation, and use `out.println()` to send the result string back to the client. (Wrap this logic in a try-catch to handle `NumberFormatException` or `ArrayIndexOutOfBoundsException` if the user types bad input, sending an “Invalid format” message back).
2. **The Server:** Create a class `CalculatorServer`.
 - In main, open a `ServerSocket` on port 9000.
 - Use an infinite `while(true)` loop to `accept()` connections.
 - For every accepted socket, instantiate your `CalcClientHandler` and start it in a `new Thread()`.
3. **The Client:** Create a class `CalcClient`.
 - Connect to “localhost” on port 9000.
 - Wrap the `InputStream` and `OutputStream`.
 - Wrap `System.in` in a `Scanner` to read keyboard input from the user.
 - Read the welcome message from the server and print it.
 - Create a `while(true)` loop:
 - (a) Prompt the user: `System.out.print("Enter math problem: ");`
 - (b) Read the user’s input using `Scanner.nextLine()`.
 - (c) Send the input to the server using `out.println()`.
 - (d) If the user typed “quit”, break the loop and exit.
 - (e) Otherwise, read the server’s response using `in.readLine()` and print it.
4. **Test in Concurrency:**
 - Start the `CalculatorServer`.
 - Open **Terminal A** and run `CalcClient`.

- Open **Terminal B** and run `CalcClient`.
- Verify that you can send math problems from *both* terminals without one blocking the other. Type ‘quit’ in one terminal to ensure the other stays connected.

5. **Thinking Question:**

- Thread creation in Java has a performance cost. If this calculator server became as popular as Google and received 100,000 connections per second, what would happen if we executed `new Thread(handler).start()` for every single user? (Research “Thread Pools” or “ExecutorService” to hint at the real-world solution).

References

- [Oracle: All About Sockets](#): The official Java documentation for working with URLs, Sockets, and Datagrams.
- [Baeldung: A Guide to Java Sockets](#): A highly practical, Java-focused guide with code examples for basic and multi-threaded server creation.
- [Oracle: Defining and Starting a Thread](#): A review of the `Runnable` interface used in Part 2 of this lab.
- [GeeksforGeeks: Socket Programming in Java](#): A beginner-friendly tutorial breaking down the Client-Server architecture with simple, copy-pasteable examples for immediate testing.